Adevar Labs

Prepared for:

M0
V2

September 04, 2025

Audit Report

Version: after fix review

Table of Contents

1. Executive Summary

Executive Summary

About M0

The M0 Extensions protocol enables the creation of yield-bearing extension tokens based on the M token. The system consists of two components:

The solana-m-extension programs deployed by stablecoin providers:

- **m_ext** provides a modular framework for creating custom extension tokens
- **ext_swap** enables direct swaps between extension tokens

The solana-m programs (**earn**, **portal**):

- **earn** manages extension tokens and yield distribution through index updates
- **portal** handles cross-chain bridging with enhanced support for direct extension token transfers

Audit Summary

The audit of the M0 Extensions protocol, which facilitates yield-bearing extension tokens on Solana, revealed no critical or high-severity issues, highlighting the team's strong development practices and security awareness.

The audit identified 7 findings (1 medium, 6 low) and 2 enhancement opportunities, with recommendations to address medium-severity issues, ensure fair yield distribution, and strengthen trust assumptions.

Key areas for improvement include addressing role inconsistencies, inbound limit bypasses, and fee distribution edge cases.

The codebase demonstrates a solid understanding of the Anchor framework, supported by an extensive test suite. Pre-launch and post-launch recommendations aim to ensure a secure and robust deployment.

- Number of Findings: 7 total
- Critical: 0
- High: 0
- Medium: 1
- Low: 6
- Number of Enhancement Opportunities: 2

Risk Profile

The following table summarizes the distribution of identified vulnerabilities by risk level:

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	1	1	0
Low	6	3	3

Overall Security Posture

Before Fix Review

The M0 team has demonstrated solid development practices and security awareness as no critical or high-severity issues were found during this audit. The codebase shows a deep understanding of the Anchor framework and Solana programs library. An extensive test suite is also present.

Specific areas warrant additional focus to further strengthen the protocol:

- **Role inconsistencies**: The earn manager role still has control over some parameters even after being removed.
- **Bridge inbound limit bypass**: The inbound limits are not checked for portal configurations with more than one transceiver.
- **Fee distribution**: Edge cases regarding fee distribution are present in the crank version of the extension.

After Fix Review

Following the fix review, the M0 team has successfully addressed the issues identified during the audit, demonstrating the team's commitment to security and their technical proficiency in implementing fixes.

- **Role inconsistencies**: Control from removed earn managers is now limited.
- **Bridge inbound limit bypass**: The inbound rate limits now works also for multi-transceiver configurations.
- **Fee distribution**: Yield distribution will be automated ensuring proper ordering of operations.

Launch Recommendations

Before Fix Review

To ensure a smooth and secure launch, we offer the following recommendations:

I. Mandatory Before Launch:

- Address the inbound rate limit bypass in multi-transceiver configurations if planning to use more than one transceiver.
- Review and document trust assumptions around operators, especially regarding non-active earn managers.

II. Strongly Recommended:

- Fix Medium severity issues when applicable.
- Ensure fair distribution of yield to earners by preventing loss of unclaimed yield.
- Review allowed Token2022 Extensions for M-Extension tokens.

III. Post-Launch Monitoring:

- Monitor Extension admins' activity to prevent $M tokens from being transferred to unauthorized parties.
- Prepare and anticipate emergency procedures to ensure rapid response capability.
- Deploy continuous fuzzing in CI/CD pipeline.

After Fix Review

All mandatory and strongly recommended recommendations have been adressed.

Audit Scope

- **Repository:** m0-foundation/solana-m-extensions/tree/scaledui-m-v2-changes
- **Commit Hash:** *d73c338026dcd823066fc83a0695be8a6e8e00de*
- **Files/Modules in Scope:** PR #38

- **Repository:** m0-foundation/solana-m/tree/scaledui-prototype
- **Commit Hash:** *eb4e792950672d6f311abdf25107b77b031965b0*
- **Files/Modules in Scope:** PR #138

2. Assumptions and Considerations

Audit Assumptions

The following limitations and constraints should be considered when reviewing this security assessment report:

I. **Earn Authority and Earn Manager are trusted**

Both roles are set by the M-Extension Admin and are expected to be trusted. The yield distribution to Earners, as well as the index update, rely on those authorities.

II. **Index synchronization**

M token multiplier is synchronized to its most recent value before claiming yield for Earners (crank feature).

III. **The Portal program is a modified Wormhole NTT Manager program**

Untouched code of the original program is assumed to function correctly. Only added/modified logic has been reviewed during the audit.

Centralization Risks

I. **M Token PermanentDelegate and FreezeAuthority**

For legal compliance reasons, the M token has both the PermanentDelegate and FreezeAuthority extensions activated, allowing the Earn program to freeze and transfer M token balances.

II. **Bridging operation pausability**

The Portal program has the ability to pause and unpause bridging operations.

III. **Admin privileges**

- Setting and modifying fees
- Whitelisting/delisting extensions and wrap authorities
- Modifying critical protocol parameters

Trust Assumptions

- **Wormhole infrastructure dependency**: The Portal program is built on Wormhole's Native Token Transfer (NTT) framework, inheriting trust assumptions about Wormhole's guardian network, message verification, and cross-chain communication reliability
- **Off-chain yield calculation**: The earn authority relies on off-chain RPC data collection and balance calculations to determine accurate yield distributions for individual earners

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Impact

Impact reflects the potential consequences of the issue–particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

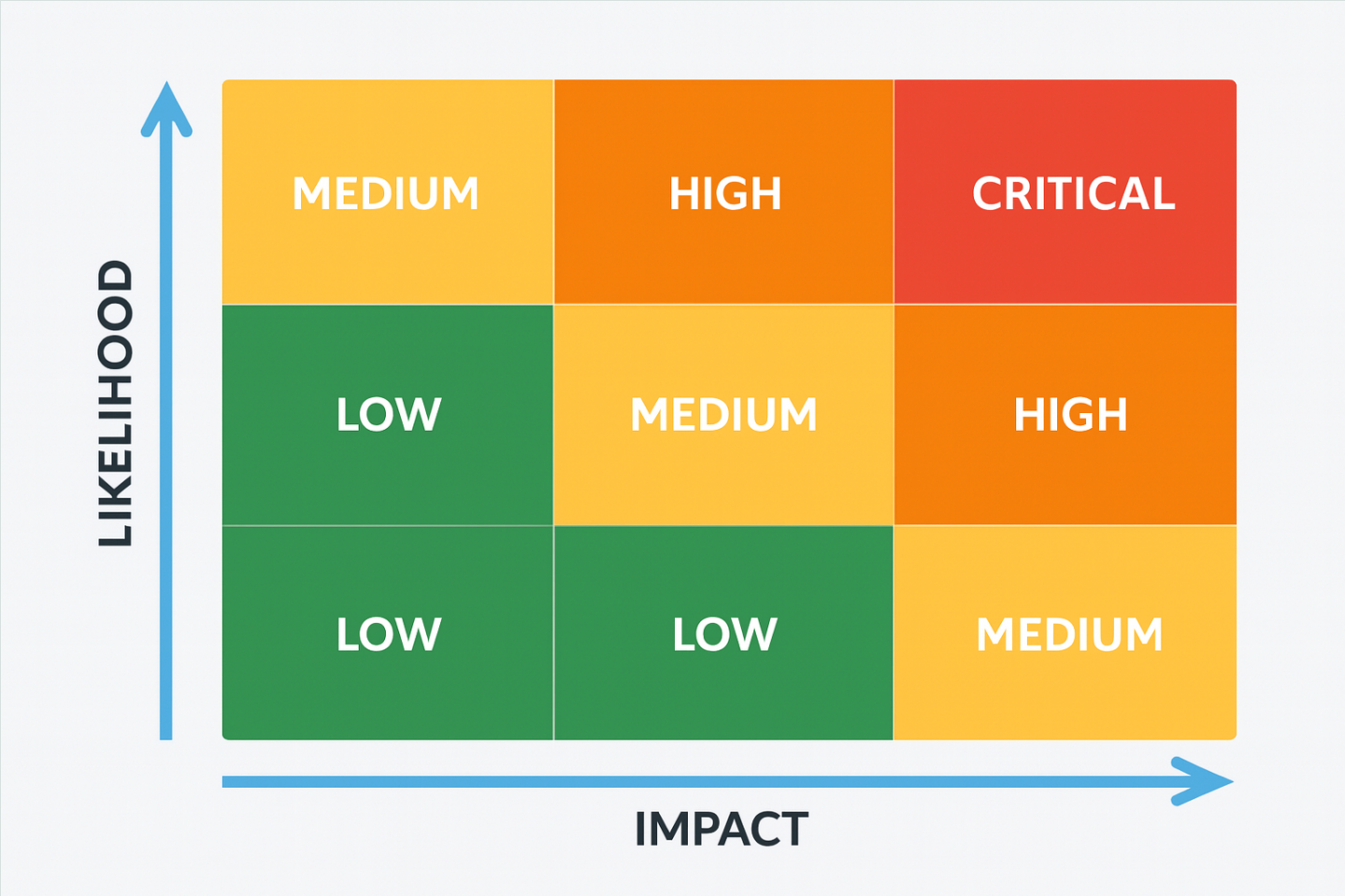
- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact + high likelihood (e.g. a bug that could allow anyone to drain a substansial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Moderate impact and/or discoverability
- **Low**: Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

M01: Inbound Rate Limit Bypassed in Multi-Transceiver Portal Configurations Due to Incorrect Amount Variable Usage

Status: Resolved

Impact:

Likelihood:

Severity: Medium

Location: https://github.com/m0-foundation/solana-m/blob/eb4e792950672d6f311abdf25107b77b031965b0/programs/portal/src/instructions/redeem.rs#L169-L186

>_ redeem.rs RUST

```
167:     accs.inbox_item.votes.set(accs.transceiver.id, true)?;
168:
169:     if accs
170:         .inbox_item
171:         .votes
172:         .count_enabled_votes(accs.config.enabled_transceivers)
173:         < accs.config.threshold
174:     {
175:         return Ok(());
176:     }
177:
178:     let release_timestamp = match accs.inbox_rate_limit.rate_limit.consume_or_delay(am
         ount) {
179:         RateLimitResult::Consumed(now) => {
180:             // When receiving a transfer, we refill the outbound rate limit with
181:             // the same amount (we call this "backflow")
182:             accs.outbox_rate_limit.rate_limit.refill(now, amount);
183:             now
184:         }
185:         RateLimitResult::Delayed(release_timestamp) => release_timestamp,
186:     };
187:
188:     accs.inbox_item.release_after(release_timestamp)?;
```

Description:

The issue is located in the Portal program's **redeem** instruction and is true for configurations where **threshold > 1**.

The **amount** variable is initialized to 0 at the start of each redeem call and is only set to the actual transfer amount during the first vote when **!accs.inbox_item.init == true**.

When the first transceiver votes, the amount is correctly extracted and stored in **inbox_item.transfer.amount**. However, if the voting threshold is not yet reached (threshold > 1), the function returns early before reaching the rate-limiting logic. The rate limit is therefore

not consulted on the first vote.

On subsequent votes from other transceivers, the initialization block is skipped and **amount** remains 0. Once the threshold is reached, the call **consume_or_delay(0)** always succeeds since consuming zero capacity from any rate limiter is trivial, regardless of the actual transfer size stored in **inbox_item.transfer.amount**. This causes large transfers to bypass rate limits entirely.

Recommendation:

Read the amount from the **inbox_item**:

DIFF

```
if accs
    .inbox_item
    .votes
    .count_enabled_votes(accs.config.enabled_transceivers)
    < accs.config.threshold
{
    return Ok(());
}

+ let transfer_amount = accs.inbox_item.transfer.amount;

- let release_timestamp = match accs.inbox_rate_limit.rate_limit.consume_or_delay(amount) {
+ let release_timestamp = match
    accs.inbox_rate_limit.rate_limit.consume_or_delay(transfer_amount) {
    RateLimitResult::Consumed(now) => {
-       accs.outbox_rate_limit.rate_limit.refill(now, amount);
+       accs.outbox_rate_limit.rate_limit.refill(now, transfer_amount);
        now
    }
    RateLimitResult::Delayed(release_timestamp) => release_timestamp,
};
```

Developer Response:

Will fix.
PR: https://github.com/m0-foundation/solana-m/pull/161

L01: Retroactive Fee Application for Crank Version of Extensions

Status: Acknowledged

Impact: Low / Medium / High

Likelihood: Low / Medium / High

Severity: Low

Location: https://github.com/m0-foundation/solana-m-extensions/blob/d73c338026dcd823066fc83a0695be8a6e8e00de/programs/m_ext/src/instructions/crank/earn_manager/configure.rs#L41-L49

>_ configure.rs RUST

```rust
    /// If the fee token account is None, it will not be updated.

    pub fn handler(ctx: Context<Self>, fee_bps: Option<u64>) -> Result<()> {
        if let Some(fee_bps) = fee_bps {
            // Validate the fee percent is not greater than 100%
            if fee_bps > ONE_HUNDRED_PERCENT_U64 {
                return err!(ExtError::InvalidParam);
            }

            ctx.accounts.earn_manager_account.fee_bps = fee_bps;
        }

        if let Some(fee_token_account) = &ctx.accounts.fee_token_account {
```

Description:

The **ConfigureEarnManager** instruction allows the earn manager to set the fee basis points (**fee_bps**) for yield distribution. Once the fee is set, it also applies to past unclaimed yield.

Recommendation:

The ConfigureEarnManager instruction allows the earn manager to set the fee basis points (fee_bps) for yield distribution.
Once the fee is set, it also applies to past unclaimed yield. A coordination of the Earn Authority and the Earn Manager is required to ensure this does not happen.

Developer Response:

Acknowledge. Earn managers will likely be associated with the earn authority and can coordinate updates to ensure the impacts of fee updates are minor. If abused by an Earn Manager, the admin can remove them.

L02: An Inactive Earn Manager Can Temporarily Redirect Earners' Yield

Status: Resolved

Impact: Low Medium High

Likelihood: Low Medium High

Severity: Low

Location: https://github.com/m0-foundation/solana-m-extensions/blob/d73c338026dcd823066fc83a0695be8a6e8e00de/programs/m_ext/src/instructions/crank/earner/set_recipient.rs#L10-L17

>_ set_recipient.rs RUST

```rust
#[derive(Accounts)]
pub struct SetRecipient<'info> {
    #[account(
        constraint =
            signer.key() == earner_account.user ||
            signer.key() == earner_account.earn_manager
            @ ExtError::NotAuthorized,
    )]
    pub signer: Signer<'info>,

    #[account(
```

Description:

When an earn manager is removed (set to inactive), they can still call the **SetRecipient** instruction to update the earner's recipient token account.
This becomes impossible only once the **RemoveOrphanedEarner** instruction has been called, creating a window of opportunity allowing a compromised earn manager to set the earner's recipient token account to one of its own addresses, in order to redirect the claimed yield.

Recommendation:

Only allow the earner or the active earn manager to call the instruction. Alternatively, but less practical, to prevent such actions, ensure the **RemoveEarnManager** instruction is followed by a **RemoveOrphanedEarner** instruction in the same transaction. The earner's yield should also be claimed before removing the orphaned earner, as otherwise its pending yield would be lost.

Developer Response:

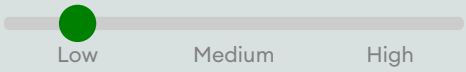
Will fix.
PR: https://github.com/m0-foundation/solana-m-extensions/pull/56

L03: Earners Will Lose Pending Yield When Removed

Status: Acknowledged

Impact:

Likelihood:



Severity: Low

Location: https://github.com/m0-foundation/solana-m-extensions/blob/d73c338026dcd823066fc83a0695be8a6e8e00de/programs/m_ext/src/instructions/crank/earn_manager/remove_earner.rs#L34-L40

>_ remove_earner.rs RUST

```rust
}

impl RemoveEarner<'_> {
    /// This instruction allows the earn manager to remove an earner.
    /// The earner account is closed and the lamport balance is recovered.

    pub fn handler(_ctx: Context<Self>) -> Result<()> {
        Ok(())
    }
}
```

Description:

Currently, the **RemoveOrphanedEarner** instruction allows anyone to remove any earner given that earner's corresponding **earn_manager** is not active (removed), without ensuring the earner has received their pending yield. There is no check to confirm that the earner has received all their earned yield, meaning an earner can be removed before their rewards are distributed. The issue is also present in **RemoveEarner** regarding lost yield, even though this function is protected and can only be called by the active earn manager of the earner.

Recommendation:

Before removing an earner, ensure their pending yield is distributed.

Developer Response:

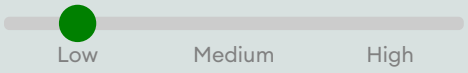
Acknowledge. We handle this with our offchain claim bot.

L04: Earn Manager Can Block Earner Yield by Setting Fee Token Account to Frozen Account

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location: https://github.com/m0-foundation/solana-m-extensions/blob/d73c338026dcd823066fc83a0695be8a6e8e00de/programs/m_ext/src/instructions/crank/earn_manager/configure.rs#L30-L31

>_ configure.rs RUST

```rust
    pub earn_manager_account: Account<'info, EarnManager>,

    #[account(token::mint = global_account.ext_mint)]
    pub fee_token_account: Option<InterfaceAccount<'info, TokenAccount>>,
}
```

Description:

An earn manager can set their fee token account to a frozen token account using the **ConfigureEarnManager** instruction. When **earn_authority** tries to claim yield for an earner, the protocol attempts to mint the fee to the manager's token account. If the account is frozen, the mint fails and the claim transaction reverts, blocking earners from receiving their yield.

Recommendation:

- In **handle_fee**, return early if the **earn_manager_token_account.state == AccountState::Frozen**
- Add a check to ensure the fee token account is not frozen before updating it

Developer Response:

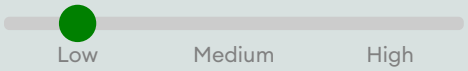
Will fix.
PR: https://github.com/m0-foundation/solana-m-extensions/pull/57

L05: Denial of Service in Migration via Donating Few M Tokens to Old M Vault Token Account

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location: https://github.com/m0-foundation/solana-m-extensions/blob/d73c338026dcd823066fc83a0695be8a6e8e00de/programs/m_ext/src/instructions/migrate.rs#L127-L129

>_ migrate.rs RUST

```
125:         let old_vault_m_amount = self.old_vault_m_token_account.amount;
126:
127:         if new_vault_m_amount < old_vault_m_amount {
128:             return err!(ExtError::InsufficientCollateral);
129:         }
130:
131:         Ok(())
```

Description:

During the migration process, the $M amount in the new vault is verified to have at least as much $M as the previous vault.

An attacker can donate a small amount of M tokens to the old vault (**old_vault_m_token_account**). Since the migration requires the new vault to have at least as much M as the old vault, this artificially increases the required amount for migration.

Depending on how the migration is configured, this opens up a possibility for a Denial of Service, as the migration can be blocked by donating a sufficient amount of M tokens to the old vault, making it impossible to satisfy the collateral requirement.

For this attack to be possible, the attacker needs to hold $M tokens, which can only be obtained by extensions admin and are able to transfer the tokens using the unwrap instruction. As admins are whitelisted participants, the attack is unlikely, as the cost to the attacker would likely outweigh the benefit. Then, once an actor is able to get $M tokens, it becomes difficult to prevent the attack, as they could disperse the tokens to numerous addresses to prevent the freezing of the tokens.

Recommendation:

Rather than comparing the amount of $M tokens from both vaults, check that the new vault has enough $M tokens to cover the $EXT supply. Alternatively, using the Token2022 TransferHook Extension could prevent transfer to unauthorized parties.

Developer Response:

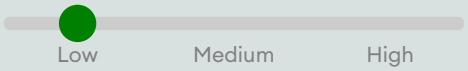
Will fix.
PR: https://github.com/m0-foundation/solana-m-extensions/pull/58

L06: ext_mint Can Have CloseMintAuthority & PermanentDelegate Extensions Activated

Status: Acknowledged

Impact:

Likelihood:



Severity: Low

Location: https://github.com/m0-foundation/solana-m-extensions/blob/d73c338026dcd823066fc83a0695be8a6e8e00de/programs/m_ext/src/instructions/initialize.rs#L59-L65

>_ initialize.rs RUST

```
57:     pub m_mint: InterfaceAccount<'info, Mint>,
58:
59:     #[account(
60:         mut,
61:         mint::token_program = ext_token_program,
62:         mint::decimals = m_mint.decimals,
63:         constraint = ext_mint.supply == 0 @ ExtError::InvalidMint,
64:     )]
65:     pub ext_mint: InterfaceAccount<'info, Mint>,
66:
67:     /// CHECK: Validated by the seeds, stores no data
```

Description:

The **Initialize** instruction for the extension mint (**ext_mint**) does **not** check for the presence of the **closeMintAuthority** or **permanentDelegate** extensions on the mint account. This means the mint could be created with these authorities set, which introduces critical risks.

Recommendation:

Add explicit checks in the **Initialize** instruction to ensure that:
- **closeMintAuthority** is not active on mint
- **permanentDelegate** is not active on mint

Developer Response:

Acknowledge. While there is a possibility of an extension owner closing the mint account and recreating with different token extensions than initialized with, I don't think it specifically makes sense to prohibit owners from adding these extensions if required for compliance reasons.

5. Enhancement Opportunities

E01: Timestamping New Index of M During Redeem Would Be More Accurate

Location: https://github.com/m0-foundation/solana-m/blob/eb4e792950672d6f311abdf25107b77b031965b0/programs/earn/src/instructions/portal/propagate_index.rs#L57-L57

>_ propagate_index.rs RUST

```
55:             // If the new index is strictly greater than the current one, update the m
        ultiplier.
56:             if new_index > current_index {
57:                 let timestamp = Clock::get()?.unix_timestamp;
58:
59:                 update_multiplier(
```

Description:

In the current state of the program, the applied timestamp to the M Mint ScaledUI extension is the current Unix timestamp when the **PropagateIndex** instruction is called. This instruction is called by the **ReleaseInbound** instruction, which can withhold a message for a configured period of time (24h in the current settings) if the Portal inbound limit is reached. This creates a timing discrepancy where the applied timestamp may be significantly later than when the actual redemption/index update should have been recorded, leading to imprecise timestamping of M token index changes.

Potential Benefit:

Users and systems relying on timestamp accuracy for yield calculations will have access to the true timing of index updates rather than delayed execution timestamps.

Recommendation:

Provide the redeem timestamp information to the **PropagateIndex** instruction.

Developer Response:

Still discussing internally. Agree this would be an improvement, but trying to gauge the whether the impact is worth the amount of refactoring required.

E02: Unused Accounts Provided in Some Instructions Context

Location: https://github.com/m0-foundation/solana-m/blob/eb4e792950672d6f311abdf25107b77b031965b0/programs/earn/src/instructions/open/add_registrar_earner.rs#L42-L42

>_ add_registrar_earner.rs RUST

```
40:     pub user_token_account: InterfaceAccount<'info, TokenAccount>,
41:
42:     pub system_program: Program<'info, System>,
43:
44:     pub token_program: Program<'info, Token2022>,
```

Description:

Some unused accounts have been identified during the review. Examples (the list is not exhaustive):

solana-m:
- **add_registrar_earner.rs:42** : **system_program** is included but not used. This account was used in the previous version of the repo but is no longer necessary.

solana-m-extension:
- **remove_orphaned_earner.rs:17** : **global_account** is included but not used.

Potential Benefit:

These unnecessary account requirements increase the transaction size and gas costs without providing any functionality.

Recommendation:

Review and remove all instructions for unnecessary accounts.

Developer Response:

Will fix.
PRs:
- https://github.com/m0-foundation/solana-m-extensions/pull/55
- https://github.com/m0-foundation/solana-m/pull/162

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification

- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

Appendix A: Appendix: Alternative Solution for Re-Frozen Account Yield

The current M0 implementation on Solana utilizes the Token2022 ScaledUiAmount extension for yield distribution through a rebasing mechanism. This creates an unintended consequence where frozen M token accounts continue to accrue yield despite being restricted from transfers.

When an earner is removed from the approved list via **remove_registrar_earner**, their frozen account still benefits from the global multiplier increases, allowing non-compliant or potentially exploited programs to accumulate protocol yield indefinitely.

Solution 1: Index-Based Tracking

Proposed Solution Overview

This proposed draft solution introduces a state-tracking mechanism at the extension level to manage yield accrual during freeze periods. By maintaining freeze and unfreeze indices, the system can calculate and exclude yield generated during restricted periods while preserving legitimate pre-freeze earnings. Tokens are burned in the **m_ext** program to ensure limited M0's control over the extension tokens.

State Additions

Three new fields are added to the **ExtGlobalV2** account:

- **frozen: bool** - Boolean flag indicating the current freeze state of the extension's M vault ATA
- **frozen_at_index: u128** - The yield index at the time of account freezing
- **unfrozen_at_index: u128** - The yield index at the time of account unfreezing

New Instruction

A new instruction **freeze_extension** in the **m_ext** program, called via Cross-Program Invocation from **add_registrar_earner** and **remove_registrar_earner**, manages these state transitions.

Implementation Details

Crank Variant Implementation

Freeze state management

- Set **frozen = true** and record **frozen_at_index = current_timestamp** when M vault ATA state is **Frozen**

- Set **frozen = false** and record **unfrozen_at_index = current_timestamp** when transitioning from frozen to **Initialized** state

Claim prevention

- Block **claimFor** operations when the M vault ATA is frozen

- On unfreeze, check if **unfrozen_at_index > last_claim_index**

- If true, use **unfrozen_at_index** for yield calculation and reset to 0

Yield calculation during freeze

The amount of M tokens to burn upon unfreezing is calculated as:

amount_to_burn = vault_m_amount * (unfrozen_at_index / frozen_at_index - 1)

Collateralization check

Before burning, ensure the extension remains adequately collateralized such as:

m_current_collateral - amount_to_burn >= ext_supply

ScaledUI Variant Implementation

The ScaledUI variant requires additional modifications:

Multiplier sync behavior

- Skip **sync_multiplier** execution while the M vault ATA is frozen

- This prevents the extension's multiplier from increasing during the freeze period

Modified multiplier calculation

- **get_latest_multiplier_and_timestamp** should use **unfrozen_at_index** in place of **cached_m_multiplier** at unfreeze time

- This ensures yield calculations exclude the frozen period

Unclaimed Yield Preservation

For yield accrued before freezing, the system can calculate it using the **last_claimed** index and the **frozen_at_index**.

This yield remains claimable after unfreezing, ensuring legitimate earnings are not lost.

Unjustified Freeze Reversion

For cases where a freeze was not justified, the system provides a reversion path:

- Account is unfrozen without burning tokens
- Both **frozen_at_index** and **unfrozen_at_index** are reset to zero
- **frozen** is set back to **false**
- Normal yield accrual resumes from the point of unfreezing

Wrap/Unwrap Operations

During freeze periods, wrap and unwrap operations may require special handling.

Considerations

1. **Burn irreversibility**: Burned tokens cannot be recovered if the freeze was later deemed unjustified
2. **Complexity**: Additional state management increases potential for implementation errors

Solution 2: Skipped Distribution

Overview

A simpler alternative achieves the same goal by decoupling yield tracking from yield distribution. This approach skips token minting/multiplier updates while maintaining index progression, then burns the accumulated excess upon unfreezing.

State Addition

Only a single field is needed in **ExtGlobalV2**:

- **frozen: bool** - Boolean flag indicating the current freeze state

No index tracking fields are required, simplifying the implementation compared to Solution 1.

Implementation Details

Crank Variant Implementation

In the **claimFor** instruction, when frozen:

- Update the earner's **last_claim_index** and **last_claim_timestamp** to current values
- Skip the actual reward minting
- This effectively marks the yield as "claimed" without distributing tokens

When unfrozen:

- burn the excess M tokens accumulated during the freeze
- proceed with normal logic.

ScaledUI Variant Implementation

In the **sync** instruction, when frozen:

- Update the internal **last_m_index** tracking
- Skip the extension's multiplier update
- This prevents yield distribution while maintaining index synchronization

Burn on Unfreeze

When unfreezing:

- Calculate excess collateral: **excess = vault_m_balance - ext_supply** (accounting for multipliers)
- Burn the excess M tokens from the vault
- Set **frozen = false**
- Resume normal operations

Considerations

- **Burn Irreversibility**: Like Solution 1, burned tokens cannot be recovered
- **Index Progression**: The **last_claim_index** advances without corresponding rewards

Comparison of Solutions

Both solutions achieve the goal of preventing frozen accounts from benefiting from yield.

Solution 2 offers a simpler implementation while achieving the same economic outcome:

- **Solution 1** provides precise index-based tracking but requires complex state management
- **Solution 2** achieves similar results with simpler logic by burning excess collateral on unfreeze

Disclaimer

This appendix presents draft proposals for managing re-frozen account yield in the M0 system. The solutions described herein are conceptual and have not been implemented or thoroughly tested. They should NOT be used as-is for production.

The mathematical formulas, implementation details, and technical approaches presented are subject to change and may contain errors or oversimplifications.

This document is intended for discussion and analysis purposes only.